RETAILMART V3 · SQL

The DISTINCT Family, Procedures & Triggers

— Siraj Sir

📱 WhatsApp Announcement

Bonus Session: The DISTINCT Family, Procedures & Triggers

You asked two great questions, so here is a full bonus class. First we finish the DISTINCT family -- including IS DISTINCT FROM, the NULL-safe comparison we only mentioned during the Data Quality day. Then we make the database work for you with stored procedures and triggers.

Part 1: The DISTINCT family -- DISTINCT, IS DISTINCT FROM, DISTINCT ON
Part 2: Stored Procedures & Functions -- reusable server-side logic
Part 3: Triggers -- the database reacts automatically

Reminder: procedures and triggers CREATE objects and change data. Run every CREATE / CALL / UPDATE example in YOUR OWN practice database (accio_NN), never against the shared RetailMart database. SELECT examples are safe to run against RetailMart.

TODAY'S AGENDA (2 Hours)

Section	Time	Topics
Part 1: The DISTINCT Family	50 min	DISTINCT -- de-duplicate rows, IS DISTINCT FROM -- NULL-safe compare, Change detection for data quality, DISTINCT ON -- one row per group
Part 2: Procedures & Functions	45 min	Function vs procedure, CREATE PROCEDURE / CALL, Parameters, variables, control flow, Transactions inside procedures
Part 3: Triggers	45 min	Trigger function + trigger, BEFORE vs AFTER, NEW vs OLD, Validation & audit patterns, When NOT to use triggers

RECAP

From the Data Quality & Deduplication Day

- We listed IS DISTINCT FROM as a NULL-safe comparison tool
- We said regular = and <> behave badly when NULLs are involved
- You asked: HOW do we actually use it? -- that is Part 1 of today
- Spoiler: it is the cleanest way to detect 'did this value really change?'

STORY

Three Skills That Separate Juniors from Seniors

A junior writes a SELECT. A senior makes the database do the heavy lifting: they compare nullable columns correctly with IS DISTINCT FROM, they pick exactly the right row with DISTINCT ON, they package repeatable work into a stored procedure, and they let triggers enforce rules automatically. All four show up in interviews and in production every week.

STORY

Three Cousins, Very Different Jobs

DISTINCT, IS DISTINCT FROM, and DISTINCT ON share a word but solve completely different problems. DISTINCT removes duplicate rows. IS DISTINCT FROM compares two values safely even when they are NULL. DISTINCT ON keeps one chosen row per group. By the end of Part 1 you will never mix them up again.

DEFINITION

DISTINCT = unique combinations

- DISTINCT looks at ALL selected columns together
- It returns each unique combination of values once
- It cannot keep extra columns 'along for the ride'
- Output order is not guaranteed without ORDER BY

Which regions and statuses exist?

```
-- Every unique (region, status) pair across all orders
SELECT DISTINCT dr.region_name, o.order_status
FROM sales.orders o
JOIN stores.stores s    ON o.store_id = s.store_id
JOIN core.dim_region dr ON s.region_id = dr.region_id
ORDER BY dr.region_name, o.order_status;

-- Count the customers who actually ordered
SELECT COUNT(DISTINCT cust_id) AS buyers
FROM sales.orders;
```

COMMON MISTAKE

The DISTINCT Trap

SELECT DISTINCT cust_id, order_date, net_total … does NOT give you one row per customer. It returns every unique (cust_id, order_date, net_total) triple -- basically every order. DISTINCT cannot 'pick the latest order per customer'. That is the job of DISTINCT ON, later in this part.

STORY

The Comparison That Silently Loses Rows

You compare yesterday's price with today's price to find what changed: WHERE new_price <> old_price. It works… until one of them is NULL. In SQL, NULL <> 5 is not TRUE -- it is NULL, which WHERE treats as 'not matched'. So every row where a value went from NULL to a number, or a number to NULL, is silently dropped. In a data-quality job, that is a disaster. IS DISTINCT FROM fixes it.

DEFINITION

IS DISTINCT FROM = 'are these two values different?', NULL-safe

- a IS DISTINCT FROM b -> TRUE when they differ, treating NULL as a normal value
- NULL IS DISTINCT FROM NULL -> FALSE (they are the same)
- NULL IS DISTINCT FROM 5 -> TRUE (they differ)
- It NEVER returns NULL -- always TRUE or FALSE. That is why it is safe in WHERE.

Run this against any database (SELECT only)

```
SELECT
  (5    =  5)                    AS eq_5_5,        -- true
  (5    =  NULL)                 AS eq_5_null,     -- NULL  (!)
  (NULL =  NULL)                 AS eq_null_null,  -- NULL  (!)
  (5    IS DISTINCT FROM 5)      AS idf_5_5,       -- false
  (5    IS DISTINCT FROM NULL)   AS idf_5_null,    -- true
  (NULL IS DISTINCT FROM NULL)   AS idf_null_null, -- false
  (NULL IS NOT DISTINCT FROM NULL)  AS indf_null;  -- true

-- Notice: = gives NULL the moment a NULL appears.
-- IS DISTINCT FROM always gives a clean true/false.
```

REVEAL

Why = Breaks in WHERE

WHERE keeps a row only when its condition is TRUE. Because NULL = NULL and NULL = 5 both evaluate to NULL (not TRUE), any comparison involving a NULL is quietly discarded. IS DISTINCT FROM resolves NULL to a real TRUE/FALSE, so those rows are evaluated correctly instead of vanishing.

Which products actually changed? (data quality)

```
-- Compare an incoming staging load to the live table.
-- We want rows where ANY tracked column truly changed,
-- including changes to or from NULL.
SELECT s.product_id
FROM staging_products s
JOIN products p ON s.product_id = p.product_id
WHERE s.price        IS DISTINCT FROM p.price
   OR s.category_id  IS DISTINCT FROM p.category_id
   OR s.supplier_id  IS DISTINCT FROM p.supplier_id;

-- With plain <> you would MISS every row where a column
-- went NULL -> value or value -> NULL. That is the bug
-- IS DISTINCT FROM exists to prevent.   (run in accio_NN)
```

Matching rows where NULL should equal NULL

```
-- 'Same email, where two NULL emails count as a match'
SELECT a.customer_id, b.customer_id
FROM customers.customers a
JOIN customers.customers b
  ON a.email IS NOT DISTINCT FROM b.email   -- NULL = NULL here
 AND a.customer_id < b.customer_id;

-- IS NOT DISTINCT FROM is the exact inverse:
--   a IS NOT DISTINCT FROM b  ==  NOT (a IS DISTINCT FROM b)
```

PRO TIP

PRO TIP

Read 'a IS DISTINCT FROM b' as 'a and b are different (and NULL counts as a value)'. Whenever you compare columns that might be NULL -- change-detection, deduplication, slowly-changing dimensions, MERGE/UPSERT conditions -- reach for IS DISTINCT FROM instead of <> and IS NOT DISTINCT FROM instead of =.

COMMON MISTAKE

IS DISTINCT FROM Pitfalls

- Using <> for change-detection on nullable columns -> silently drops NULL transitions
- Forgetting it is the WHOLE operator: it is 'IS DISTINCT FROM', not 'DISTINCT FROM'
- Expecting an index to be used -- it often cannot, so do not put it on huge hot paths blindly
- Confusing it with DISTINCT or DISTINCT ON -- different tools entirely

Practice 1: Customers Whose Tier Changed

EASY

SCENARIO: A nightly load writes new customer tiers into a staging table. The data team needs the customers whose tier genuinely changed -- including those who went from no tier (NULL) to a tier.

TASK: Compare customers.customers (live) with a customer_staging table on customer_id and return every customer whose tier changed, NULL-safe.

HINT: Join on customer_id, filter with IS DISTINCT FROM on the tier column.

Answer

✅ ANSWER

```
SELECT c.customer_id,
       c.tier      AS old_tier,
       st.tier     AS new_tier
FROM customers.customers c
JOIN customer_staging st
  ON c.customer_id = st.customer_id
WHERE c.tier IS DISTINCT FROM st.tier;
-- catches Gold->Platinum AND NULL->Bronze AND Gold->NULL
```

Practice 2: Flag Real Price Changes

MEDIUM

SCENARIO: Finance wants an audit of products whose price OR cost actually moved between two daily snapshots, ignoring rows that only look different because of NULLs.

TASK: Given price_snapshot_old and price_snapshot_new (product_id, price, cost_price), return product_id plus old/new values for any row where price or cost_price changed, NULL-safe.

HINT: Two IS DISTINCT FROM conditions joined with OR.

Answer

✅ ANSWER

```
SELECT n.product_id,
       o.price       AS old_price, n.price       AS new_price,
       o.cost_price  AS old_cost,  n.cost_price  AS new_cost
FROM price_snapshot_new n
JOIN price_snapshot_old o ON n.product_id = o.product_id
WHERE n.price      IS DISTINCT FROM o.price
   OR n.cost_price IS DISTINCT FROM o.cost_price;
```

STORY

Keep the Row You Choose

DISTINCT removes duplicate rows. DISTINCT ON does something only PostgreSQL can: it keeps the FIRST row for each value of a key, and YOU decide which row is 'first' with ORDER BY. It is the cleanest way to answer 'the latest order per customer' or 'the top store per region'.

DEFINITION

DISTINCT ON (cols) = first row per group

- DISTINCT ON (expr) keeps the FIRST row for each value of expr
- 'First' is decided by ORDER BY -- so ORDER BY is mandatory
- ORDER BY must START with the same columns as DISTINCT ON
- You keep all the other columns of that chosen row

DISTINCT ON pattern (safe on RetailMart)

```
SELECT DISTINCT ON (cust_id)
       cust_id,
       order_id,
       order_date,
       net_total,
       order_status
FROM sales.orders
ORDER BY cust_id,            -- group key first (required)
         order_date DESC,    -- then 'latest' wins
         order_id DESC;      -- tiebreaker for same-day orders
```

REVEAL

DISTINCT ON vs ROW_NUMBER()

DISTINCT ON is shorter and very fast for 'top-1 per group'. The ROW_NUMBER() window approach is more flexible -- it handles top-N per group and exposes the rank. Use DISTINCT ON for top-1; reach for ROW_NUMBER() when you need top-3, ties, or per-group numbering.

Same result, window-function style

```
SELECT cust_id, order_id, order_date, net_total
FROM (
  SELECT *,
         ROW_NUMBER() OVER (
           PARTITION BY cust_id
           ORDER BY order_date DESC, order_id DESC
         ) AS rn
  FROM sales.orders
) t
WHERE rn = 1;
```

COMMON MISTAKE

DISTINCT ON Pitfalls

- Forgetting ORDER BY -> you get an arbitrary row per group
- ORDER BY not starting with the DISTINCT ON columns -> ERROR
- No tiebreaker -> non-deterministic result when the sort key ties
- It is PostgreSQL-specific -- other databases do not have it

Practice 3: Most Recent Review Per Product

MEDIUM

SCENARIO: The catalog team wants the single most recent review for each product to show on the product page.

TASK: Using customers.reviews, return one row per product_id: the latest review by review_date, with its rating and review_text.

HINT: DISTINCT ON (product_id) with ORDER BY product_id, review_date DESC.

Answer

✅ ANSWER

```
SELECT DISTINCT ON (product_id)
       product_id,
       review_id,
       review_date,
       rating,
       review_text
FROM customers.reviews
ORDER BY product_id, review_date DESC, review_id DESC;
```

STORY

Stop Copy-Pasting the Same 40 Lines

Every Monday you run the same multi-step refresh: clear a summary table, recompute it from raw orders, log that it ran. Pasting that into your SQL client every week is how mistakes happen. A stored procedure packages those steps into one named command you (or a schedule) just CALL.

DEFINITION

They are not the same thing

- FUNCTION: returns a value, called inside a query with SELECT. Cannot manage transactions.
- PROCEDURE: performs actions, called with CALL. CAN COMMIT / ROLLBACK inside it.
- Use a function to compute and return data; use a procedure to DO a unit of work.
- Both take parameters and can hold full plpgsql logic (variables, IF, loops).

CREATE FUNCTION -- run in your accio_NN DB

```
-- Returns how many delivered orders a customer has
CREATE OR REPLACE FUNCTION delivered_order_count(p_cust INT)
RETURNS BIGINT
LANGUAGE sql AS $$
  SELECT COUNT(*) FROM orders
  WHERE cust_id = p_cust AND order_status = 'Delivered';
$$;

-- Call it inside a normal query:
SELECT cust_id, delivered_order_count(cust_id) AS delivered
FROM customers
WHERE cust_id IN (101, 102, 103);
```

CREATE PROCEDURE -- run in your accio_NN DB

```
-- Rebuilds a daily sales summary table in one command
CREATE OR REPLACE PROCEDURE refresh_daily_sales()
LANGUAGE plpgsql AS $$
BEGIN
  DELETE FROM daily_sales_summary;

  INSERT INTO daily_sales_summary (sale_date, orders, revenue)
  SELECT order_date, COUNT(*), SUM(net_total)
  FROM orders
  WHERE order_status = 'Delivered'
  GROUP BY order_date;

  COMMIT;   -- procedures can control transactions
END;
$$;

CALL refresh_daily_sales();   -- run the whole job
```

A procedure with an argument and a variable

```
CREATE OR REPLACE PROCEDURE archive_old_orders(p_before DATE)
LANGUAGE plpgsql AS $$
DECLARE
  moved INT;
BEGIN
  INSERT INTO orders_archive
  SELECT * FROM orders WHERE order_date < p_before;

  GET DIAGNOSTICS moved = ROW_COUNT;
  DELETE FROM orders WHERE order_date < p_before;

  RAISE NOTICE 'Archived % orders', moved;
  COMMIT;
END;
$$;

CALL archive_old_orders(DATE '2024-01-01');
```

REVEAL

Control Flow You Can Use Inside plpgsql

- Variables: DECLARE v INT; … v := 10;
- Branching: IF … THEN … ELSIF … ELSE … END IF;
- Loops: FOR r IN SELECT … LOOP … END LOOP;
- Messages: RAISE NOTICE / RAISE EXCEPTION for logging and errors

COMMON MISTAKE

Procedure & Function Gotchas

- Calling a procedure with SELECT -> error. Use CALL.
- Trying to COMMIT inside a function -> error. Only procedures manage transactions.
- RETURNS without a return path in plpgsql -> runtime error.
- Forgetting OR REPLACE means the second run fails with 'already exists'.

REAL WORLD

Where Analysts Meet Procedures

Scheduled refreshes (nightly summary tables), ETL steps, data-archival jobs, and 'recompute this report' buttons are almost always stored procedures behind the scenes. Even if you do not write them daily, you will read, debug, and call them constantly.

Practice 4: A Monthly Refresh Procedure

MEDIUM

SCENARIO: Finance wants a monthly_revenue table they can rebuild on demand and trust.

TASK: In your accio_NN DB, write a procedure refresh_monthly_revenue() that clears monthly_revenue and repopulates it with month, order count, and delivered revenue grouped by month. Then CALL it.

HINT: DATE_TRUNC('month', order_date); LANGUAGE plpgsql; BEGIN … END; COMMIT.

Answer

✅ ANSWER

```
CREATE OR REPLACE PROCEDURE refresh_monthly_revenue()
LANGUAGE plpgsql AS $$
BEGIN
  DELETE FROM monthly_revenue;
  INSERT INTO monthly_revenue (month, orders, revenue)
  SELECT DATE_TRUNC('month', order_date)::date,
         COUNT(*), SUM(net_total)
  FROM orders
  WHERE order_status = 'Delivered'
  GROUP BY DATE_TRUNC('month', order_date);
  COMMIT;
END;
$$;

CALL refresh_monthly_revenue();
```

STORY

Who Changed This Price?

A product's price quietly changed and nobody knows who did it or when. You could ask everyone to remember to log changes -- they will forget. Or you make the database log it AUTOMATICALLY, every single time, with a trigger. Triggers are how databases enforce rules and keep audit trails without trusting humans to remember.

DEFINITION

Trigger = code that fires on a table event

- A trigger runs automatically on INSERT, UPDATE, or DELETE
- It calls a special trigger FUNCTION that RETURNS trigger
- BEFORE triggers can modify or reject the row; AFTER triggers react to it
- Inside the function: NEW = the incoming row, OLD = the previous row

Trigger function -- run in your accio_NN DB

```
-- Logs every REAL price change into a price_audit table.
-- IS DISTINCT FROM makes it NULL-safe (ties Part 1 together!)
CREATE OR REPLACE FUNCTION log_price_change()
RETURNS TRIGGER
LANGUAGE plpgsql AS $$
BEGIN
  IF NEW.price IS DISTINCT FROM OLD.price THEN
    INSERT INTO price_audit(product_id, old_price, new_price, changed_at)
    VALUES (OLD.product_id, OLD.price, NEW.price, now());
  END IF;
  RETURN NEW;   -- AFTER trigger: value ignored but required
END;
$$;
```

CREATE TRIGGER

```
CREATE TRIGGER trg_price_audit
AFTER UPDATE ON products
FOR EACH ROW
EXECUTE FUNCTION log_price_change();

-- Now ANY price update is logged automatically:
UPDATE products SET price = 1299 WHERE product_id = 42;
-- price_audit gets a new row -- no extra code needed.
```

REVEAL

Choosing the right timing

- BEFORE INSERT/UPDATE: validate or auto-fill values (set NEW.updated_at = now())
- AFTER INSERT/UPDATE/DELETE: react -- write audit logs, update summary tables
- INSERT has only NEW. DELETE has only OLD. UPDATE has both.
- A BEFORE trigger can RETURN NULL to silently cancel the operation

Stamp timestamps and guard data -- accio_NN DB

```
CREATE OR REPLACE FUNCTION stamp_and_check()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
BEGIN
  NEW.updated_at := now();                 -- auto-fill
  IF NEW.price < 0 THEN                     -- validate
    RAISE EXCEPTION 'price cannot be negative';
  END IF;
  RETURN NEW;   -- BEFORE trigger: returned row is saved
END;
$$;

CREATE TRIGGER trg_products_guard
BEFORE INSERT OR UPDATE ON products
FOR EACH ROW EXECUTE FUNCTION stamp_and_check();
```

COMMON MISTAKE

Trigger Dangers

- OLD is not available on INSERT, NEW is not available on DELETE -> NULL errors
- A trigger that UPDATEs the same table can fire itself -> infinite recursion
- Heavy logic in a row trigger runs once PER ROW -> can cripple bulk updates
- Hidden side effects: people forget a trigger exists and get 'mystery' rows

INTERVIEW

Q: When would you use a trigger over application code?

A: When the rule must hold no matter who writes to the table -- multiple apps, manual SQL, bulk loads. A trigger enforces it at the database level, so the audit log or validation can never be bypassed or forgotten. The trade-off is that triggers are invisible side effects: document them, and avoid heavy per-row logic on bulk-write tables.

Practice 5: Auto Stock-Out Flag

HARD

SCENARIO: Operations wants inventory rows flagged the instant stock hits zero, with no application change.

TASK: In your accio_NN DB, write a BEFORE UPDATE trigger on inventory that sets NEW.status to 'Out of Stock' when NEW.quantity_on_hand = 0, and 'Normal' otherwise.

HINT: BEFORE UPDATE FOR EACH ROW; modify NEW.* and RETURN NEW -- no separate UPDATE.

Answer

✅ ANSWER

```
CREATE OR REPLACE FUNCTION set_stock_status()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
BEGIN
  NEW.status := CASE WHEN NEW.quantity_on_hand = 0
                     THEN 'Out of Stock' ELSE 'Normal' END;
  RETURN NEW;
END;
$$;

CREATE TRIGGER trg_stock_status
BEFORE UPDATE ON inventory
FOR EACH ROW EXECUTE FUNCTION set_stock_status();
```

KEY TAKEAWAYS

DISTINCT de-duplicates whole rows: It returns unique combinations of all selected columns -- it cannot pick a specific row per group.

IS DISTINCT FROM is NULL-safe: Use it (not <>) to compare nullable columns. It is the correct tool for change-detection, dedup and MERGE conditions.

DISTINCT ON picks one row per group: ORDER BY is mandatory and must start with the DISTINCT ON columns. Perfect for latest-per-group.

Functions return, procedures act: SELECT a function for a value; CALL a procedure for a unit of work. Only procedures manage transactions.

Triggers run automatically on table events: BEFORE to validate/auto-fill, AFTER to react. NEW and OLD give you the rows. Use them with discipline.

INTERVIEW

Four questions you can now answer cold

- Why does WHERE a <> b drop rows, and how does IS DISTINCT FROM fix it?
- When does DISTINCT ON beat ROW_NUMBER()? (top-1 per group, concise & fast)
- Difference between a procedure and a function? (CALL vs SELECT; transactions)
- How would you build an audit trail without changing application code? (AFTER trigger)

Keep Practicing

In your own accio_NN database, rebuild today's examples from memory: a change-detection query with IS DISTINCT FROM, a DISTINCT ON 'latest per group' query, one refresh procedure, and one audit trigger. If you can write those four cold, you are ahead of most candidates in interviews.